



SEMESTER PROJECT

Timetable Management System

Deliverable 1 (Client-Side Architecture)

BS Software Engineering Blue 6th

Roll no: 086

Name: Muhammad Basit

TOTAL GRADE POINT: 10

GRADE POINT ALLOCATED:

ASSESSOR'S COMMENTS

Abstract:

This report documents the client-side architecture of the Smart University Timetable Management System, developed as Deliverable 1 of the semester project for the Department of Software Engineering, University of Sialkot. The frontend is implemented using Vue 3 and Vite, and demonstrates Single Page Application (SPA) routing, a modular component hierarchy, and inter-component communication through typed props and custom events (\$emit). The system supports three user roles — Student, Teacher, and Advisor — each with role-specific dashboards and provides a shared weekly timetable view showing scheduled and free lecture periods across an eight-semester departmental structure. This document covers the system architecture, routing design, component structure, and inter-component data flow, supported by code excerpts and interface screenshots.

1. Introduction:

University departments frequently need to reschedule lectures due to teacher unavailability, room conflicts, or unforeseen events. Manually coordinating such changes between students, teachers, and the departmental advisor is time-consuming and error prone. The Smart University Timetable Management System addresses this by providing a centralized, role-based web application through which students can view their timetable and request a reschedule, teachers can review and act on those requests, and the advisor can oversee the full departmental timetable, manage academic entities, and receive notifications of all activity.

This deliverable focuses exclusively on the client-side of the system, built with Vue 3 and Vite. Backend integration (Express.js and MongoDB) already exists as a REST API and will be formally documented in a later deliverable; here it is treated as a service the frontend consumes.

2. System Overview:

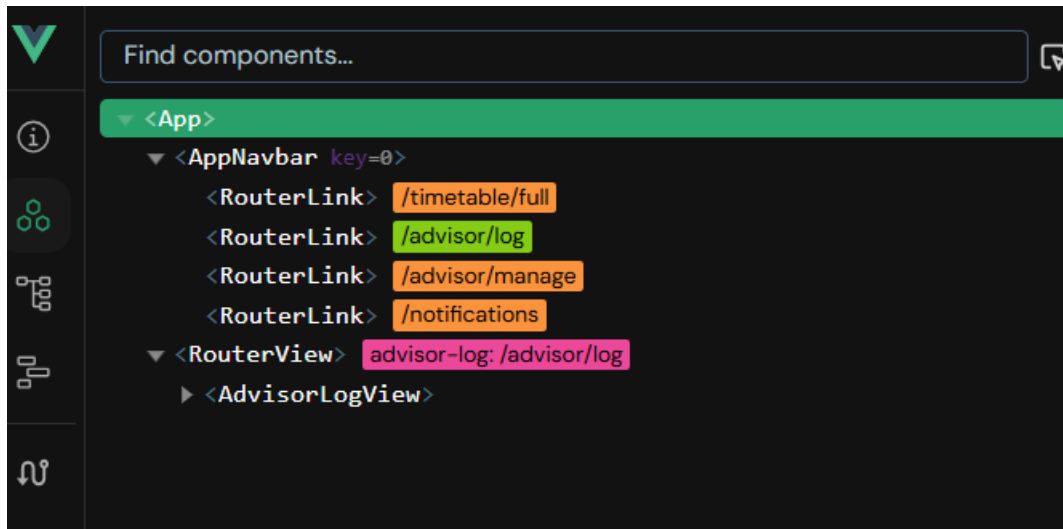
The system defines three user roles, each restricted to its own set of views by a client-side navigation guard:

- **Student:** View their personal weekly timetable and the full departmental timetable, submit reschedule requests by selecting a free period, and track the status of past requests.
- **Teacher:** Reviews pending reschedule requests from students, approves or rejects them with an optional comment, and views a history of processed requests.
- **Advisor:** Views the full departmental timetable and activity log, and manages semesters, sections, subjects, teachers, and timetable slots. Only the advisor may add teachers or assign subjects/slots to the timetable; students and teachers may only propose or respond to reschedule requests, and the advisor is notified whenever a change occurs.

The academic structure spans eight semesters (BSCS-1 through BSCS-8), each with its own section. Each teaching day is divided into seven lecture periods of 70 minutes each, running from 8:30 AM to 4:40 PM, Monday through Friday. On Friday, a 20–30-minute Jummah break is inserted after the third period, shifting the remaining four lectures later in the day. Classes are mostly held in these rooms (106, 107, SJ Lab, 102 Lab, AK Lab, 311, Phar-2, 108, 201). Periods with no assigned class are shown as free — the same information used to compute valid reschedule targets.

3. Client-Side Architecture:

The frontend follows a layered Vue 3 architecture where the root App component serves as the main entry point. It contains a navigation component (AppNavbar) with RouterLinks for navigating between different pages and a RouterView that dynamically renders the active view based on the current route. Each route loads its corresponding view component, and the interface is built from reusable, modular components to improve maintainability and scalability. Network communication is handled through a centralized API client module, keeping UI components focused on presentation and user interaction.



4. Single Page Application (SPA) Routing:

Navigation is handled entirely on the client using Vue Router in HTML5 history mode. Each screen is registered as a named route, and role restrictions are attached as route metadata (meta.role). A global before Each navigation guard checks the current authentication state and the target route's required role before allowing navigation, redirecting unauthenticated users to the login page and redirecting users who attempt to access a route outside their role back to their own home screen. The root App.vue component renders the persistent sidebar and a <RouterView /> outlet, so switching routes swaps only the main content; no full page reload occurs. Code Listing 1 shows the router configuration, including the shared /timetable/full route (available to all three roles) and the guard logic.

```
import { createRouter, createWebHistory } from 'vue-router'
import { useAuth } from '../composables/useAuth'

const routes = [
  { path: '/login', name: 'login',
    component: () => import('../views/LoginView.vue') },

  { path: '/student/timetable', name: 'student-timetable',
    component: () => import('../views/student/StudentTimetableView.vue'),
    meta: { role: 'student' } },

  { path: '/timetable/full', name: 'timetable-full',
    component: () => import('../views/TimetableFullView.vue'),
    meta: { requiresAuth: true }, // shared: student + teacher + advisor

  { path: '/advisor/manage', name: 'advisor-manage',
```

```

    component: () => import('../views/advisor/AdvisorManageView.vue'),
    meta: { role: 'advisor' } },

]

const router = createRouter({ history: createWebHistory(), routes })

router.beforeEach((to) => {
  const { isLoggedIn, user } = useAuth()
  if (to.name === 'login' && isLoggedIn()) return roleHome()
  if (to.name !== 'login' && !isLoggedIn()) return { name: 'login' }
  if (to.meta.role && to.meta.role !== user.value?.role) return roleHome()
  return true
})

export default router

```

The root layout consumes this router through the standard `<RouterLink>` and `<RouterView>` elements, as shown in Code Listing 2.

```

<!-- src/App.vue -->
<template>
  <div class="app-shell">
    <AppNavbar v-if="isLoggedIn() && route.name !== 'login'" :user="user" />
    <main class="app-main">
      <RouterView />
    </main>
  </div>
</template>

```

5. Modular Component Architecture:

The UI is decomposed into a directory structure that separates route-level views from reusable presentational and functional components, following standard Vue conventions:

WEB PROJ:TIMETABLE	front
▼ frontend	4
▼ src	37
▼ api	38
JS client.js	39
▼ assets	40
uos-logo.png	41
▼ components	42
> admin	43
> common	44
> layout	45
> requests	46
▼ composables	47
JS useAuth.js	48
> router	49
▼ views	50
> advisor	51
> student	52
> teacher	53
▼ LoginView.vue	54
▼ NotificationsView.vue	55
▼ App.vue	56

This structure keeps each file focused on one responsibility: views own data-fetching and page-level state, while components are largely presentational and receive their data as props. The TimetableGrid component,

for example, is reused unchanged across all three role-specific timetable displays, and the Data Table component is reused across five different list views (student requests, teacher history, activity log, etc.), avoiding duplicated markup.

6. Inter-Component Communication: Props and Emits

Data flows from parent to child through typed props, and events flow back from child to parent through custom emitted events, keeping child components decoupled from how their data is fetched or how their events are ultimately handled. Code Listing 4 shows the AppNavbar component, which receives the logged-in user object as a validated prop and emits a logout event that the parent uses to redirect back to the login screen.

```
<!-- src/components/layout/AppNavbar.vue -->
<script setup>
const props = defineProps({
  user: {
    type: Object,
    required: true,
    validator: (u) => typeof u.name === 'string' && typeof u.role === 'string',
  },
})

const emit = defineEmits(['logout'])

function handleLogout() {
  logout()
  emit('logout')      // notify parent the user has logged out
  router.push({ name: 'login' })
}
</script>
```

As a second example, RequestRow, demonstrates a child sending structured payload data upward: a teacher approves or rejects a request from inside the row component, which emits the action and the relevant identifiers rather than calling the API itself. This keeps RequestRow purely presentational and reusable, while the parent view retains control over what happens on approval or rejection, as shown in Code Listing 5.

```
<!-- src/components/requests/RequestRow.vue -->
const props = defineProps({
  request: { type: Object, required: true },
})
const emit = defineEmits(['approve', 'reject'])

function confirmReject() {
  emit('reject', { id: props.request.id, comment: comment.value })
}

<!-- template -->
<button @click="emit('approve', request.id)">Approve</button>
<button @click="isRejecting = true">Reject</button>
```

The parent view (TeacherPendingView) listens for these events on the component tag itself and performs the corresponding API call, then refreshes the list — a clean separation of presentation (child) from data-fetching/business logic (parent), as shown in Code Listing 6.

```
<!-- src/views/teacher/TeacherPendingView.vue -->
<RequestRow
  v-for="r in pending" :key="r.id"
  :request="r"
  @approve="handleApprove"
  @reject="handleReject"
/>
```

7. Shared Weekly Timetable View:

A key feature of this deliverable is a single, reusable TimetableGrid component that renders the department's seven-period daily schedule and is shown, unmodified, to all three roles. It receives the flat list of scheduled slots and the period-time configuration as props, and computes which periods are occupied versus free for the currently selected day. Users move between the five weekdays using day tabs or previous/next arrows; on Friday, the grid inserts a visible JummaH-break row between the third and fourth lecture and shifts the remaining periods later, matching the department's actual Friday schedule. Free periods are rendered with an italic “Free” label, so students and teachers can quickly identify valid slots for a reschedule request, and the advisor can identify open periods when assigning new classes through the Manage screen.

8. Interface Screenshots:

The following screenshots demonstrate the application running locally on localhost, evidencing the routing, role-based views, and props/emits interactions described above.

Figure 1. Login screen

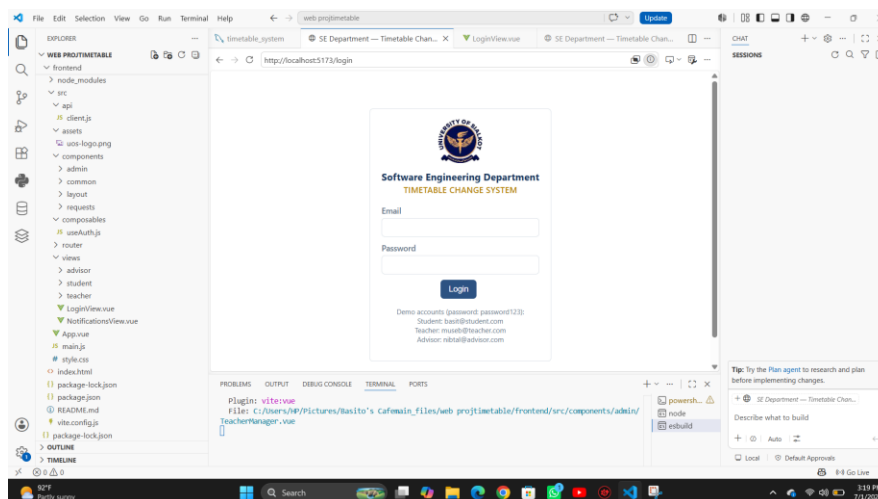


Figure 2. Student: Full Timetable view

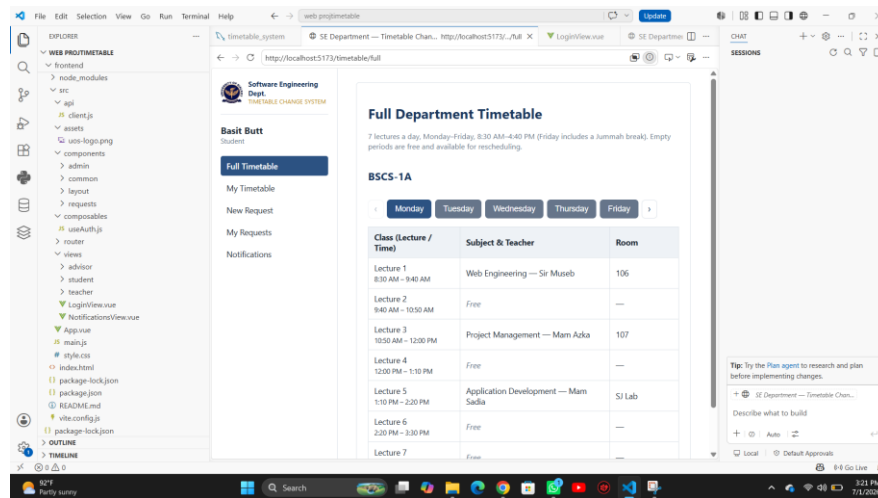


Figure 3. Teacher: Pending reschedule requests.

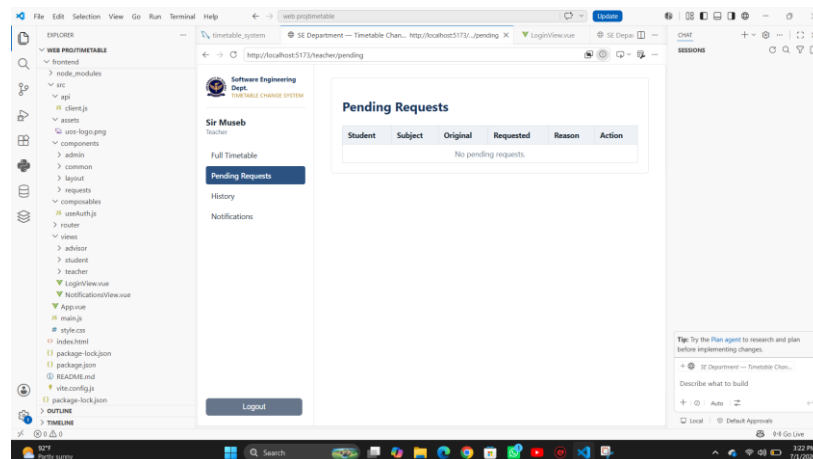
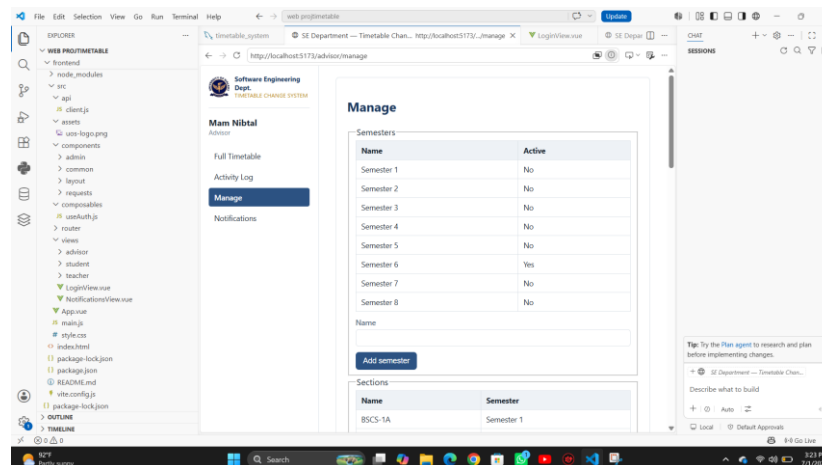


Figure 4. Advisor: Manage screen



9. Conclusion:

This deliverable establishes a complete, working client-side foundation for the Smart University Timetable Management System using Vue 3 and Vite. The application demonstrates SPA routing with role-based access control, a modular and reusable component hierarchy, and clear inter-component communication via typed props and custom events. A shared seven-period weekly timetable component, accessible to all three roles, forms the backbone of the scheduling workflow. Backend integration via Express.js and MongoDB — already functional and consumed through a centralized API client in this deliverable, will be documented formally in the next phase of the project.